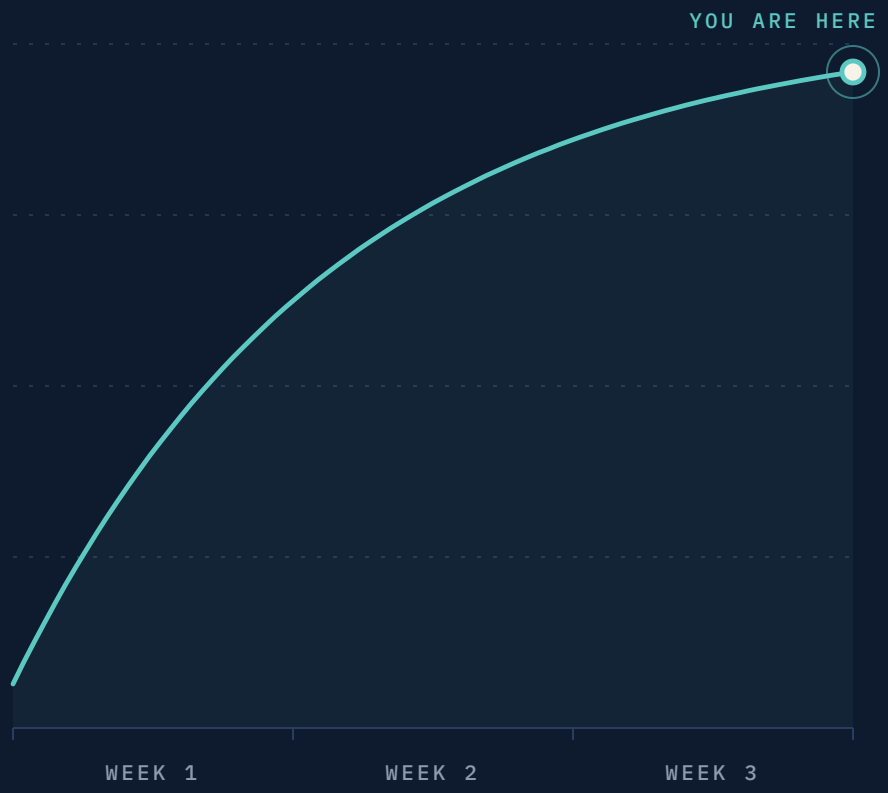
WEEK 3 · DAY 5 · OVERVIEW

Course 3 Capstone



Created by **Jeff Lenamon**

ABOUT 40 MIN / 12 SECTIONS

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Brief

The vendor's extract for July to early October is in, with a cover note. Before it touches our history, prove it against the note and the market's own calendar, fix what is wrong with a decision you can defend, and put the fixes in one function so tomorrow's file is one call. Reconcile it to the source files with nothing unexplained. Then append it and give me the snapshot for 2 October, and for 30 September from the same command. Keep both days in our database and write me one page. Describe it: no view on any rate or spread.

ABOUT 40 MIN / 12 SECTIONS

Your Route Through Day 5

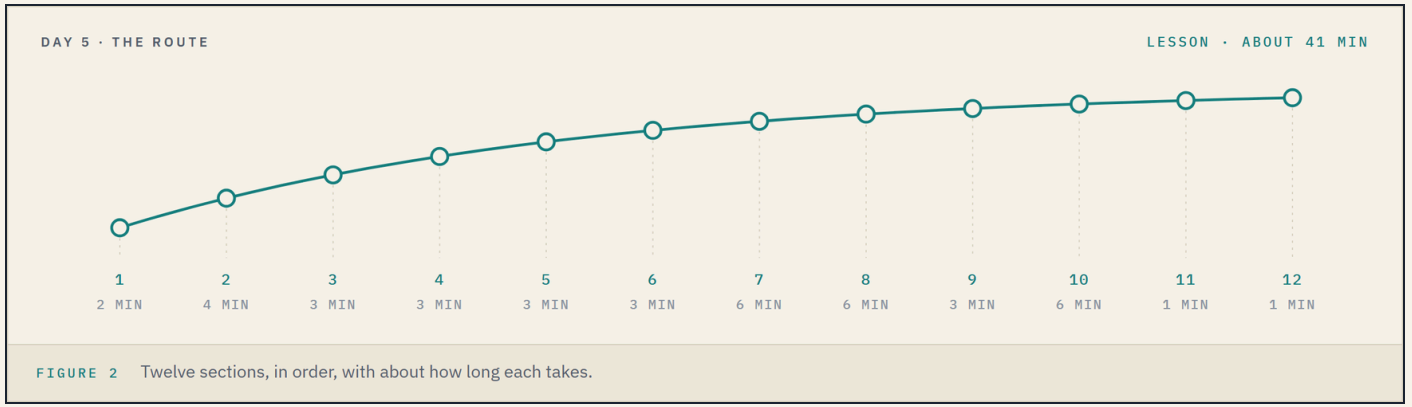


FIGURE 2 Twelve sections, in order, with about how long each takes.

	SECTION	ABOUT
1	The File and Its Note	2 min
2	Test the Extract	4 min
3	Fix the Dates and the Rows	3 min
4	Fix the Values	3 min
5	Package It	3 min
6	The Reconciliation	3 min

	SECTION	ABOUT
7	Append, Then the Snapshot	6 min
8	Rerun, Do Not Redo	6 min
9	Live, With Your Own Key (Optional)	3 min
10	The Deliverables	6 min
11	Bring It Home	1 min
12	Before You Hand In	1 min

Time. About 41 minutes for the lesson.

Read the extract against its cover note.

1 The File and Its Note

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
1.1 Rows, Columns, and a Working Copy	What shape is `raw` (**answer_1_1**)? Copy it to `extract`.
1.2 The File Against Its Note	Print `headline()` as received. How many rows beyond the note's market days (**answer_1_2a**)? Are the columns in the note's order? How many of the last row's six values differ from the note (**answer_1_2b**)?

THE VENDOR'S COVER NOTE

```
To: Credit desk
Subject: Daily extract, 2026-07-01 to 2026-10-02
Period:        2026-07-01 to 2026-10-02
Market days:   66
Last values sent, for 2026-10-02:
```

THE CHECK BEFORE AND AFTER EACH FIX

```
def headline(label, table):
    empty = int((table == '.').sum().sum() + table.isna().sum().sum())
    print(f'{label}: {len(table)} rows, {empty} cells with no number')
```

2 Test the Extract

THE SEVEN RULES

RULE	WHAT THE DESK EXPECTS
1	one row per market day, no row a copy
2	every date written `YYYY-MM-DD`
3	every date a market day
4	every market day a row
5	no value missing
6	the note's units: yields in percent, spreads in bp
7	no value stuck for 5 rows in a row

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
A date read as `YYYY-MM-DD`	`format='%Y-%m-%d', errors='coerce'`	`=DATE(LEFT(A2,4),MID(A2,6,2),RIGHT(A2,2))`
`.` read as no value	`== '.'`	`=IFERROR(VALUE(B2),NA())`
A 5 Yr yield above 25	`> 25`	`=IFERROR(VALUE(C2),NA())>25`
A spread below 10	`< 10`	`=IFERROR(VALUE(F2),NA())<10, =IFERROR(VALUE(G2),NA())<10`

Clean every date, then drop the bad rows.

3 Fix the Dates and the Rows

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
3.1 Dates and Copies	Read every date in the form it was written, then remove the exact copies. How many rows are left (**answer_3_1**)?
3.2 The Row of Dots	One row holds `.` in every value. Give its date as text year-month-day (**answer_3_2a**), and whether `source` has a row for it, `True` or `False` (**answer_3_2b**). Decide and fix.
3.3 Rules 3 and 4: The Market's Own Days	Test the dates against `source`'s: give the date that is not a market day (**answer_3_3a**) and the market day with no row (**answer_3_3b**), as text. `md.missing_weekdays(...).intersection(source.index)` finds the second.

THE FILES

FILE	WHAT IT IS
`course3_capstone_extract.csv`	the vendor's extract, read as text
`course3_capstone_cover_note.txt`	the vendor's note
`treasury_par_yields.csv`	the Treasury par curve
`course3_synthetic_spreads.csv`	two synthetic spreads
`holdings.csv`	the September book, synthetic

4 Fix the Values

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
4.1 Rule 6: Units	Set the dates as the index and read the values as floats. Fix the units by 2.4's thresholds. Give the largest yield (**answer_4_1a**) and the smallest spread (**answer_4_1b**) after the fix.
4.2 Rule 7: A Stale Run	Run `md.stale_runs(..., min_length=5)` on each column. Which column has a run (**answer_4_2a**), and on what date does it start, as text (**answer_4_2b**)?
4.3 Restore From the Source	Restore the repeats and the missing market day from `source`, sort by date, and `assert` the dates equal the period's market days in `source`. Give the restored day's 10 Yr (**answer_4_3**).
Checkpoint: The Cleaned Extract	Does `extract` match `source` and the expected decisions? Every line must read `correct` before section 5.

The extract now equals the source on every market day. The first sitting ends here.

5 Package It

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
5.1 **`clean_extract`**, Docstring First	In **`clean_extract.py`**, written with **`%%writefile`** in a cell of its own, write **`clean_extract(raw, source)`**: a docstring listing your seven decisions, then the decisions in that order, its inputs unchanged.
5.2 Commit It	Commit **`clean_extract.py`** alone, with a subject naming the deliverable, and show the history.

THE RULES

- **Work in a copy:** fixes go to **`extract`**, never to **`raw`**.
- **State every decision** in a markdown cell, with **`headline()`** before and after the fix.
- **The market calendar is the Treasury file's own dates;** never fill a value from a neighbour.
- **Describe:** never forecast, rank, or recommend.
- **Restart and Run All** before you hand in.

One call rebuilds the cleaned extract from the file as it arrived.

6 The Reconciliation

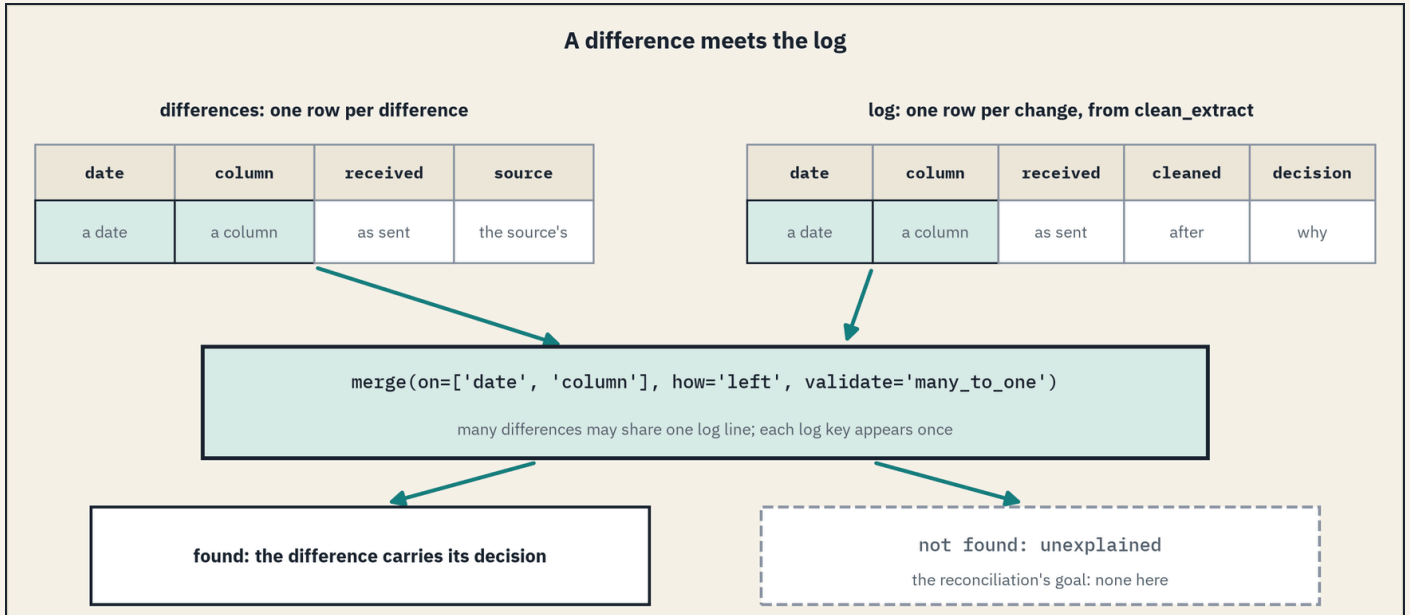


FIGURE 3 Differences join the log on date and column: one decision each, and none left unexplained.

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
6.1 The File Against the Source	Read `raw`'s dates in both forms and fix nothing else. Build **differences** (`date` as text, `column`, `received`, `source`): a row per extra copy, per row on a day with no market, per market day with no row (column `all`), and per value that differs from `source`. How many (**answer_6_1**)?
6.2 A Decision for Every Difference	Merge `differences` with `log` on `date` and `column`, `validate='many_to_one'`, as **explained**. How many have no decision (**answer_6_2a**), and how many decisions do they use (**answer_6_2b**)?

Every difference has a decision behind it. The second sitting ends here.

7 Append, Then the Snapshot

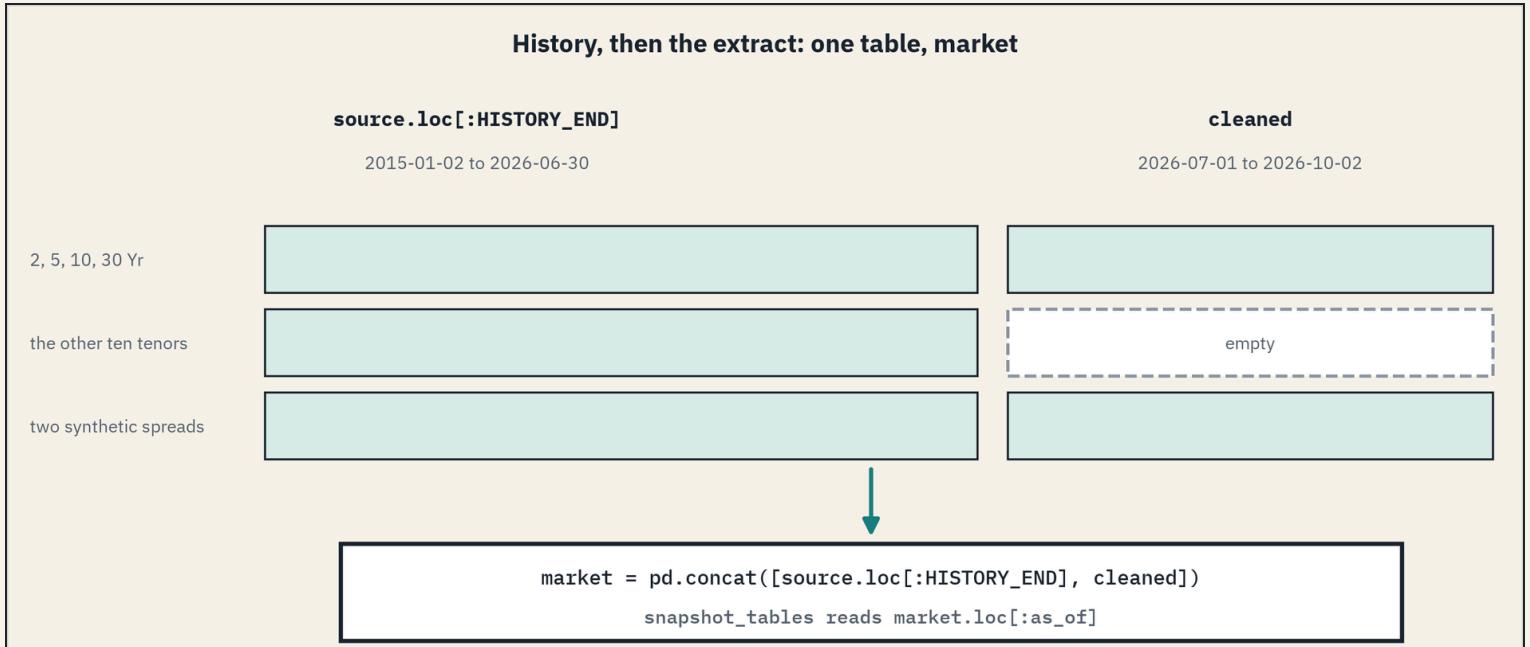


FIGURE 4 The 2015 to 2026-06-30 history and the 2026-07-01 to 2026-10-02 extract stack into one market table.

THE QUESTIONS

QUESTION
7.1 The Appended Market
7.2 The Snapshot for 2 October
7.3 The Book on Four Tenors
7.4 `snapshot.py`, Rewritten
Read the Diff, Then Commit

The snapshot says what it cannot show, and the script that writes it is in the record.

8 Rerun, Do Not Redo

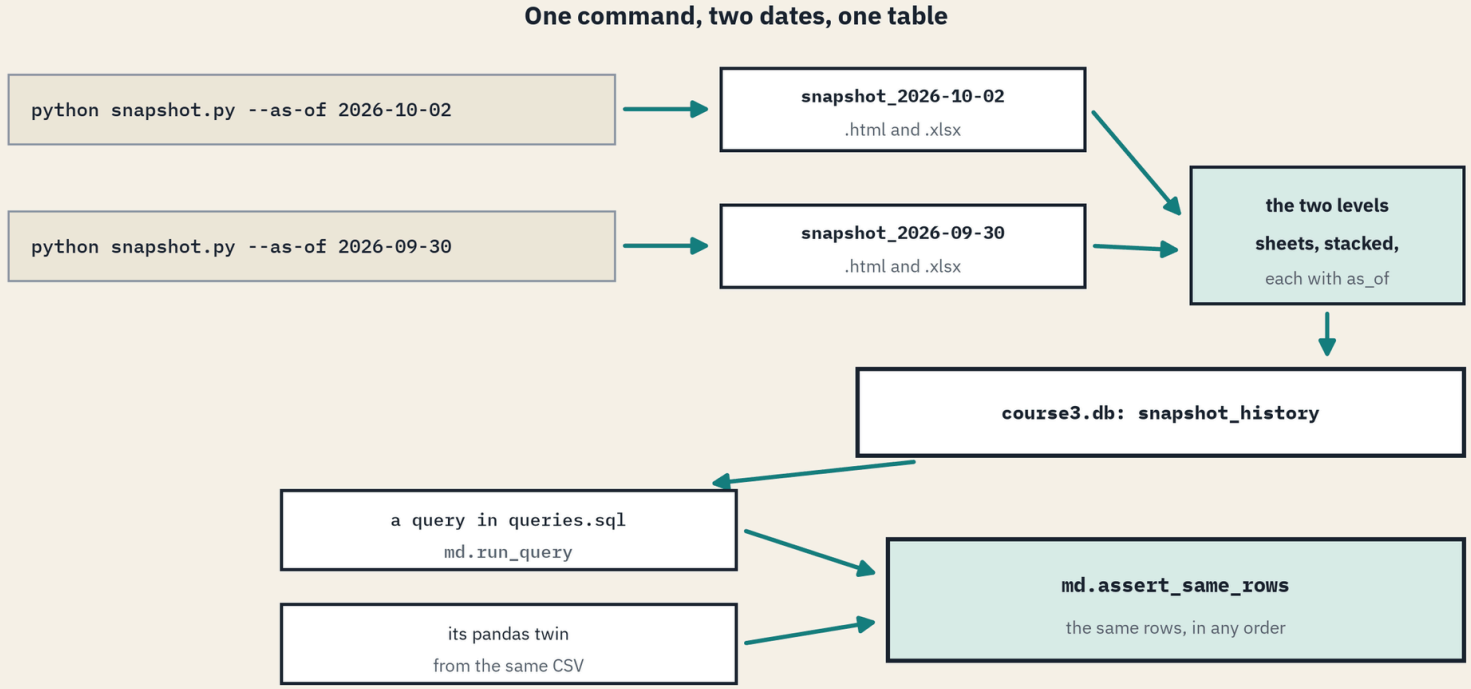


FIGURE 5 One command runs 2026-10-02 and 2026-09-30, and both levels sheets stack into snapshot_history.

THE QUESTIONS

QUESTION
8.1 Two Dates, One Command
8.2 The Two Levels Side by Side
8.3 Both Days in the Database
8.4 A Test, Then Commit

Both days are in SQL, and a test says the market equals its sources.

9 Live, With Your Own Key (Optional)

THE LIVE CELL

```
live_ids = {'DGS2': '2 Yr', 'DGS5': '5 Yr', 'DGS10': '10 Yr', 'DGS30': '30 Yr'}
live_curve = pd.DataFrame({live_column: md.fred_series(live_id, fred_key, start='2026-07-01',
                                                       end='2026-10-02')
                           for live_id, live_column in live_ids.items()}).dropna()
live_common = live_curve.index.intersection(source.index)
live_gap = (live_curve.loc[live_common] - source.loc[live_common, YIELDS]).abs().max().max()
assert len(live_common) > 0 and live_gap <= 1e-9, 'FRED and the source files disagree'
print(f'{len(live_common)} dates in both, largest gap {live_gap:.1e} percent')
```

WHAT A KEY SHOWS

The saved run had no key, so both print the skip message. With a key, the first prints the dates FRED and the source files share, and a gap of zero. The second prints each licensed series' rows and dates, never a value: the snapshot uses the synthetic spreads because it is shared.

10 The Deliverables

THE ASSISTANT'S CODE

```
def complete_days(extract, source):
    days = pd.bdate_range(extract.index[0], extract.index[-1], name='date')
    return extract.reindex(days).ffill()
```

WHAT THE SECTION ASKS

QUESTION	WHAT IT ASKS
10.2 The AI-Use Log	Write one row per question you asked an AI assistant; **`none used`** is valid. Run the cell to write **`ai_use_log.md`**.
10.3 The Page	Write the page for the desk lead, 300 to 450 words, in five parts:

THE PAGE'S FIVE PARTS

- the levels on both dates, with units;
- the changes to 2 October over a day, a week and a month;
- where each series sits in its year;
- the book's DV01 and what-if;
- the data: each problem, its rows, its decision.

Bring It Home

THE COMMANDS

```
# the work folder: use the name your setup cell printed
$work = "$env:TEMP\pfi_course3_15_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\clean_extract.py", "$work\snapshot.py", "$work\test_capstone.py" .
Copy-Item "$work\course3_capstone_extract.csv" .
python -m pytest -q test_marketdata.py test_capstone.py
python snapshot.py --as-of 2026-09-30
git add clean_extract.py snapshot.py test_capstone.py
git commit -m "Course 3 capstone: clean_extract, snapshot.py, and their tests"
git tag course3
```

pytest shows 57 passed, and the 30 September page is in `snapshots`.

Before You Hand In

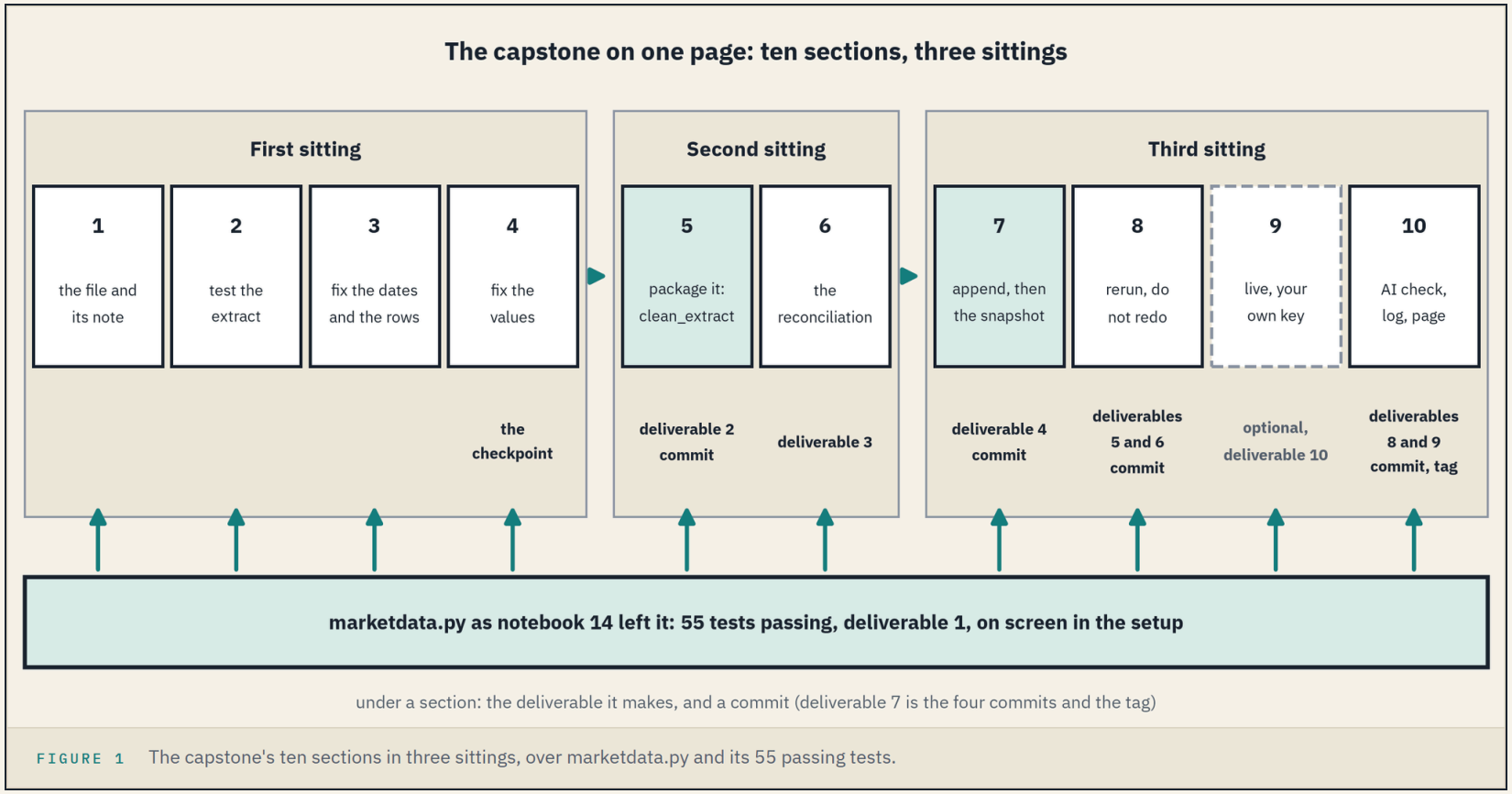


FIGURE 1 The capstone's ten sections in three sittings, over marketdata.py and its 55 passing tests.

Hand in this notebook run from the top, every check `correct`, with four commits, the tag `course3`, the log and the page.

You Can Now

In notebook 01 your module was one function and four tests; today it is 29 functions and 55 tests, each measure with its window and units named, and each tested.

Be proud: you took a file nobody vouched for, proved every row, and handed back a page the desk can check line by line.

Next is Course 4, Machine Learning: models fitted to bond data, with the same habit of proving every number.

NEXT
WEEK 1 · DAY 1

The ML Workflow and Simple Regression

Build three simple regressions of a bond's spread and judge each one on bonds it has never seen.